

Scholaria — Production Roadmap

Audit Date: 2026-08-20

Stack: Django 4.2 · DRF · PostgreSQL + pgvector · React + Redux · Docker · Groq API

Goal: Ship a live, portfolio-ready, production-grade LMS with role-based RAG.

Current State Summary

Area	Status
Core LMS (courses, modules, lessons, quizzes, assignments)	Done
Auth (JWT, role-based routes, frontend guards)	Done
LLM-optimized serializers & endpoints (<code>/api/llm/</code>)	Done
Text chunking utility (<code>utils/chunking.py</code>)	Done
<code>DocumentChunk</code> model with pgvector HNSW index	Done
<code>rag/rag.py</code> — <code>embed_input()</code>	Done
<code>rag/rag.py</code> — <code>search()</code> function	Empty
<code>rag/ingest.py</code> — ingestion pipeline	Wiped/Empty
<code>utils/embeddings.py</code> — uses OpenAI, not HuggingFace	Inconsistent
<code>courses/services/process_lessons.py</code> — calls <code>embed_data()</code> but it's broken	Broken
Django signal to auto-embed on lesson save	Missing
RAG query API endpoint (<code>/api/rag/query/</code>)	Missing
AI Chat widget (frontend)	Missing
Teacher AI Quiz Generator (frontend)	Missing
Celery / background task queue	Missing
Gunicorn in Dockerfile (production server)	Missing
<code>pgvector</code> extension in docker-compose postgres image	Wrong image
Environment variables for Groq / HuggingFace	Not in <code>.env.example</code>
Test suite	Placeholder only
<code>requirements.txt</code> (Dockerfile uses it but it doesn't exist)	Missing

Phase 1 — RAG Pipeline (Core Feature) **HIGHEST PRIORITY**

Everything else depends on this working correctly.

1.1 — Fix the Embeddings Inconsistency

Problem: `utils/embeddings.py` uses `OpenAIEmbeddings` while `rag/rag.py` uses `HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')`. Your `DocumentChunk` and `Lesson.embedding` are both `VectorField(dimensions=384)` — which is the `all-MiniLM-L6-v2` dimension. OpenAI embeddings are 1536 dimensions. They are **incompatible**.

- [] Delete the `OpenAIEmbeddings` import from `utils/embeddings.py`
- [] Rewrite `embed_data()` to use `HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')`
- [] Make it accept a single `str` or `list[str]` and return a single vector or list of vectors accordingly
- [] Add `HUGGINGFACE_CACHE_DIR` to settings to prevent re-downloading the model on every cold start

1.2 — Build the Ingest Pipeline

Problem: `rag/ingest.py` is now empty. The `courses/services/process_lessons.py` calls `embed_data()` on `lesson.content` (a string), but `embed_data` is broken (Step 1.1) and it stores to `Lesson.embedding` only — it does **not** chunk the text or populate `DocumentChunk`.

- [] Rewrite `rag/ingest.py` to be the single source of truth for ingestion:

```
# Intended logic:
def ingest_lesson(lesson: Lesson):
    # 1. Chunk the lesson content
    chunks = chunk_text(lesson.content)

    # 2. Delete old chunks for this lesson (for re-ingestion on update)
    DocumentChunk.objects.filter(
        object_id=lesson.id,
        content_type=ContentType.objects.get_for_model(Lesson)
    ).delete()

    # 3. Embed each chunk and save to DocumentChunk
    for chunk_text_item in chunks:
        vector = embed_data(chunk_text_item) # returns single vector
        DocumentChunk.objects.create(
            content=chunk_text_item,
            embedding=vector,
            course_id=lesson.module.course_id,
            content_type=ContentType.objects.get_for_model(Lesson),
            object_id=lesson.id,
            content_type_name='lesson',
        )
```

- [] Write a one-time Django management command `python manage.py ingest_all_lessons` to backfill existing lessons
- [] Delete `courses/services/process_lessons.py` (its role is now `rag/ingest.py`)

1.3 — Add Django Signal for Auto-Ingestion

Problem: There is no automatic trigger when a teacher creates or updates a lesson.

- [] In `courses/apps.py` , register a `post_save` signal on `Lesson`
- [] The signal handler calls `ingest_lesson(instance)` from `rag/ingest.py`
- [] Guard with `if instance.content:` to skip empty lessons

```
# courses/signals.py
from django.db.models.signals import post_save
from django.dispatch import receiver
from .models import Lesson

@receiver(post_save, sender=Lesson)
def on_lesson_saved(sender, instance, **kwargs):
    if instance.content:
        from rag.ingest import ingest_lesson
        ingest_lesson(instance)
```

Note: This runs synchronously for now. When you add Celery later (Phase 2), move this to a background task.

1.4 — Complete `rag/rag.py` — The Search Function

Problem: `search()` is empty with a dangling `CosineDistance()` call.

- [] Complete the `search()` function with role-scoped pgvector query:

```

from django.db.models import Q
from pgvector.django import CosineDistance
from rag.models import DocumentChunk
from courses.models import Course

def search(user, query: str, course_id: int = None, top_k: int = 3):
    query_vector = embed_input(query)

    # 1. Scope to courses this user is authorized to see
    allowed_courses = Course.objects.filter(
        Q(teacher=user) | Q(student=user) | Q(student_classes__students=user)
    ).values_list('id', flat=True)

    chunks = DocumentChunk.objects.filter(course_id__in=allowed_courses)

    # 2. Optionally narrow to a specific course
    if course_id:
        chunks = chunks.filter(course_id=course_id)

    # 3. Order by cosine distance, return top_k
    return chunks.order_by(CosineDistance('embedding', query_vector))[:top_k]

```

- [] Add a `generate_answer(context_chunks, question, user_role)` function that calls Groq:
- Student system prompt: "You are a helpful tutor. Answer using only the provided lesson context. Cite the source lesson."
- Teacher system prompt: "You are an assistant for course creators. Use the provided lesson context to generate structured quiz questions."

1.5 — Build the RAG Query API Endpoint

- [] Create `rag/views.py` with a `RAGQueryView(APIView)`
- [] Request body: `{ "question": str, "course_id": int (optional) }`
- [] Response: `{ "answer": str, "sources": [{"lesson_title": str, "content_preview": str}] }`
- [] Apply standard `IsAuthenticated` permission — role scoping is handled inside `search()`
- [] Create `rag/urls.py` and register under `api/rag/` in `scholaria/urls.py`

- [] Add `rag` to `INSTALLED_APPS` in `settings.py`

Phase 2 — Infrastructure & Correctness HIGH PRIORITY

2.1 — Fix `requirements.txt` (Dockerfile is Broken)

Problem: `Dockerfile` runs `pip install -r requirements.txt` but this file **does not exist** in the project. The Docker build will fail.

- [] Generate `requirements.txt` from `uv.lock`: `uv export --no-hashes > requirements.txt`
- [] Add `requirements.txt` to `.gitignore` as a note to regenerate, OR commit it
- [] Alternatively, change `Dockerfile` to use `uv` directly (cleaner)

2.2 — Fix the Docker Compose PostgreSQL Image

Problem: `docker-compose.yml` uses `postgres:14` which does **not** include the `pgvector` extension. Your app will crash on startup because `CREATE EXTENSION vector` will fail.

- [] Change the image to: `pgvector/pgvector:pg14`
- [] Add a healthcheck to postgres so the backend waits for it to be ready:

```
postgres:
  image: pgvector/pgvector:pg14
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5
  backend:
    depends_on:
      postgres:
        condition: service_healthy
```

2.3 — Fix the Dockerfile for Production

Problem: `CMD ["python3", "manage.py", "runserver", ...]` is Django's **development server**. It is single-threaded and explicitly unsafe for production.

- [] Add `gunicorn` to `requirements.txt`
- [] Change the CMD: `CMD ["gunicorn", "scholaria.wsgi:application", "--bind", "0.0.0.0:8000", "--workers", "4"]`
- [] Add a separate `docker-compose.prod.yml` that overrides dev settings

2.4 — Add Missing Environment Variables

Problem: `.env.example` has no entries for `GROQ_API_KEY`, HuggingFace, or any AI-related secrets. A new developer (or a deployment platform) cannot configure the app.

- [] Add to `.env.example` :
`GROQ_API_KEY=gsk_... HUGGINGFACEHUB_API_TOKEN=hf_... # optional, model runs locally`
- [] Add `GROQ_API_KEY` to `settings.py` as `GROQ_API_KEY = os.getenv("GROQ_API_KEY")`
- [] Update `rag/rag.py` to read from `settings.GROQ_API_KEY` instead of `os.environ.get()`

2.5 — Add Celery for Background Embedding (Optional but Recommended)

Problem: Embedding generation with `all-MiniLM-L6-v2` takes ~200-500ms per lesson. Running it synchronously inside a `post_save` signal will make the lesson save API endpoint slow for teachers.

- [] Add `celery` and `redis` to dependencies
- [] Add `redis` service to `docker-compose.yml`
- [] Create `scholaria/celery.py` to configure Celery
- [] Convert the `post_save` signal handler to dispatch a `ingest_lesson.delay(lesson.id)` Celery task

Phase 3 — Frontend AI Features MEDIUM PRIORITY

3.1 — Student AI Tutor Chat Widget

- [] Create `frontend/src/components/AIChatDrawer.jsx` — a slide-in panel
- [] Show it on the `LessonPage` route, pre-scoped to the current `course_id`
- [] State: `messages[]`, `isLoading`, `isOpen`
- [] On submit: POST to `/api/rag/query/` with `{ question, course_id }`
- [] Display the `sources` array as clickable lesson links below the answer
- [] Add to Redux: a `ragSlice.js` with `askQuestion` async thunk

3.2 — Teacher AI Quiz Generator

- [] On the `CreateQuizPage`, add an "Generate with AI" section
- [] Input: select a lesson from the course's lessons; click "Generate 5 questions"
- [] POST to `/api/rag/query/` with a teacher-specific prompt requesting JSON output
- [] Parse the returned structured JSON and pre-fill the quiz question/choices form
- [] Show a loading spinner and allow the teacher to edit before saving

Phase 4 — Testing MEDIUM PRIORITY

The `quizzes/tests/` directory exists but every test file is a placeholder. `courses/tests.py` and `users/tests.py` are also empty.

4.1 — Backend Unit Tests

- [] **Auth tests:** login with correct/incorrect credentials; JWT refresh; role-protected endpoint returns 403 for wrong role
- [] **Course tests:** teacher can create/update/delete; student cannot; enrollment works correctly
- [] **RAG search test:** mock the embeddings, assert that `search()` only returns chunks from authorized courses
- [] **Ingest test:** assert that saving a Lesson with content creates `DocumentChunk` rows; saving without content does not

4.2 — API Integration Tests

- [] Use DRF's `APITestCase` to test the full request/response cycle for:
- `/api/rag/query/` — returns 200 with answer and sources
- `/api/llm/courses/` — teacher sees their courses, student sees enrolled courses
- `/api/llm/courses/<id>/` — returns 403 for unauthorized course

Phase 5 — Deployment FINAL STEP

5.1 — Choose a Hosting Platform

Platform	Best For	Estimated Cost
Railway	Easiest, supports Docker + managed Postgres	~\$5/month
Render	Free tier available, PostgreSQL add-on	Free → \$7/month
Fly.io	More control, good for Docker	Free tier available

Recommendation: Railway — it natively supports Docker Compose, managed PostgreSQL with `pgvector`, and has a one-click deploy from GitHub.

Managed Postgres on Railway/Render does NOT have `pgvector` by default. You must either use their `pgvector`-enabled plans or run your own postgres container (your existing `docker-compose.yml` approach with the fixed image works for this).

5.2 — Pre-Deployment Checklist

- [] Set `DEBUG=False` in production `.env`
- [] Generate a strong `SECRET_KEY` (use `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`)
- [] Set `ALLOWED_HOSTS` to your production domain
- [] Configure `CORS_ALLOWED_ORIGINS` to your frontend domain only
- [] Set `SECURE_SSL_REDIRECT=True` if behind HTTPS (Railway/Render provides this)

- [] Run `python manage.py collectstatic` in your Dockerfile build step
- [] Run `python manage.py migrate` as an entrypoint command (not in CMD)
- [] Run `python manage.py ingest_all_lessons` after first deploy to backfill embeddings

5.3 — Frontend Deployment

- [] Create a `frontend/Dockerfile` for production (build Vite → serve with Nginx)
- [] Update `VITE_API_BASE_URL` in frontend to point to production backend URL
- [] Deploy frontend to **Vercel** (free, zero config for Vite/React) — serves separately from backend
- [] Set the production backend URL as an environment variable in Vercel

5.4 — Final Live Demo Checklist

- [] Register as a Teacher → create a Course → add a Module → add a Lesson with rich text content
- [] Verify the lesson's embedding is generated (check `DocumentChunk` table has rows)
- [] Register as a Student → enroll in the course → open the lesson
- [] Open the AI Chat drawer → ask a question about the lesson → verify the answer cites the correct lesson
- [] As Teacher → navigate to Create Quiz → use "Generate with AI" → verify questions are generated from lesson content

Summary: Order of Execution

Week 1: Phase 1 (RAG Pipeline) — Steps 1.1 → 1.2 → 1.3 → 1.4 → 1.5

Week 2: Phase 2 (Infrastructure) — Steps 2.1 → 2.2 → 2.3 → 2.4

Week 3: Phase 3 (Frontend AI) — Steps 3.1 → 3.2

Week 4: Phase 4 (Tests) + Phase 5 (Deploy)